

Experiment 02 | Depth First Search (DFS)

[Uninformed Search](#) · [Graph Traversal](#) · [docs/dfs.md](#) · [src/dfs.py](#)

Aim

To implement Depth First Search in Python and use it to traverse a graph by exploring as far as possible along each branch before backtracking.

Theory

Depth First Search is an uninformed traversal algorithm that dives deep into a graph along a single path before backtracking to explore alternatives. It is naturally expressed using recursion (the call stack) or an explicit stack, and does not guarantee the shortest path the way BFS does.

Algorithm

- Start at the source node and mark it as visited.
- Recursively visit an unvisited neighbor of the current node.
- If a node has no unvisited neighbors, backtrack to the previous node.
- Repeat until the goal is found or all reachable nodes are visited.

Python Implementation

```
def dfs(graph, node, goal, visited=None, path=None):
    if visited is None:
        visited = set()
    if path is None:
        path = []

    visited.add(node)
    path = path + [node]

    if node == goal:
        return path

    for neighbor in graph.get(node, []):
        if neighbor not in visited:
            result = dfs(graph, neighbor, goal, visited, path)
            if result is not None:
                return result

    return None
```

Sample Input

```
graph = {  
    "A": ["B", "C"],  
    "B": ["A", "D"],  
    "C": ["A", "E"],  
    "D": ["B", "E"],  
    "E": ["C", "D"],  
}  
start_node = "A"  
goal_node = "E"
```

Output

```
Running DFS from 'A' to 'E'...
```

```
Path found:
```

```
A -> B -> D -> E
```

Complexity Analysis

Metric	Complexity	Notes
Time	$O(V + E)$	Every vertex and edge is visited at most once
Space	$O(V)$	Bounded by the maximum recursion / stack depth

Applications

- Maze and puzzle solving
- Topological sorting of directed acyclic graphs
- Cycle detection in graphs
- Game tree and decision tree exploration